

Assignment 8 of MATP4820

Izaak King

April 20, 2026

Problem 1

Consider the nonnegative quadratic program:

$$\underset{\mathbf{x} \in X}{\text{minimize}} f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top \mathbf{Q} \mathbf{x} - \mathbf{c}^\top \mathbf{x}, \text{ s.t. } \mathbf{A} \mathbf{x} = \mathbf{b} \quad (1)$$

where $\mathbf{Q}$ is a symmetric and positive semidefinite matrix, and $X = \{\mathbf{x} \in \mathbb{R}^n : x_i \geq 0, i = 1, \dots, n\}$.

Use the instructor's provided file `ALM_qp.m` to write a Matlab function `ALM_qp` with input $\mathbf{Q}, \mathbf{c}, \mathbf{A}, \mathbf{b}$, initial vector $\mathbf{x}_0$, stopping tolerance `tol`, and a constant penalty parameter β . Also test your function by running the provided test file `test_ALM_qp.m` and compare to the instructor's function. In the test code, the instructor's quadratic penalty function is also called. Print your code and the results you get. Make observations between the performance of quadratic penalty method and the ALM.

Augmented Lagrangian Method for Quadratic Programming

```
function [x, hist_obj, hist_res] = ALM_qp(Q,c,A,b,tol,beta,x0)
% Augmented Lagrangian method for the quadratic programming
% min_x 0.5*x'*Q*x - c'*x
% s.t.   x >= 0, A*x == b

x = x0;

% compute the residual for the constraint A*x == b
r = A*x - b;

res = norm(r);
```

```

v = zeros(size(b)); % initialize the multiplier
grad_err = 1;
hist_res = res;
hist_obj = 0.5 * x' * Q * x - c' * x ;

while res > tol || grad_err > tol
    % use constant stepsize (based on Lipschitz constant)
    alpha = 1/ norm(Q + beta * (A' * A));

    % compute the gradient
    grad = Q * x - c + A' * v + beta * A' * (A * x - b);

    % compute violation of optimality condition
    grad_err = inf;
    while grad_err > tol
        % update x
        x = max(0, x - alpha * grad);
        % compute the gradient
        grad = Q * x - c + A' * v + beta * A' * (A * x - b);

        % compute violation of optimality condition
        grad_err = norm(x - max(0, x - grad));
    end
    % compute the residual
    r = A * x - b;
    res = norm(r);
    obj = 0.5 * x' * Q * x - c' * x;

    % save res and obj
    hist_res = [hist_res; res];
    hist_obj = [hist_obj; obj];

    % update multiplier

    v = v + beta * r;

```

end
end

Code Results: `test_ALM_qp.m` Output

```
>> test_ALM_qp
```

Minimum found that satisfies the constraints.

Optimization completed because the objective function is non-decreasing in feasible directions, to within the value of the optimality tolerance, and constraints are satisfied to within the value of the constraint tolerance.

<stopping criteria details>

Student solver: Total running time of student code is 4.7591

Instructor penalty solver: Total running time of instructor code is 101.4867

iter	dual_res	primal_res
------	----------	------------

1	1.00e-03	1.20e+00
---	----------	----------

2	1.00e-03	1.24e-02
---	----------	----------

3	1.00e-03	2.13e-04
---	----------	----------

Instructor ALM solver: Total running time of instructor code is 4.3411

Student Figure Result

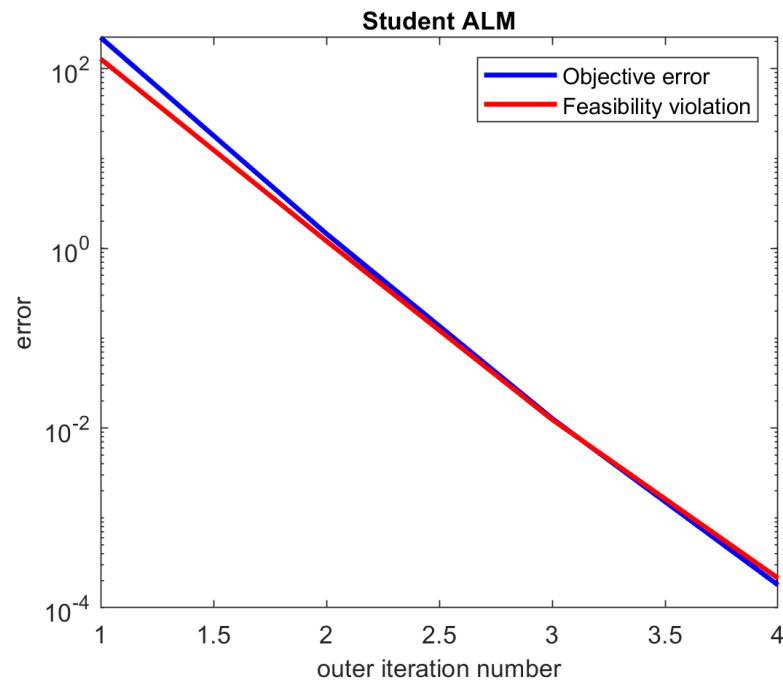


Figure 1: Student test_ALM_qp.m Result

Instructor Quadratic Penalty Test Figure Result

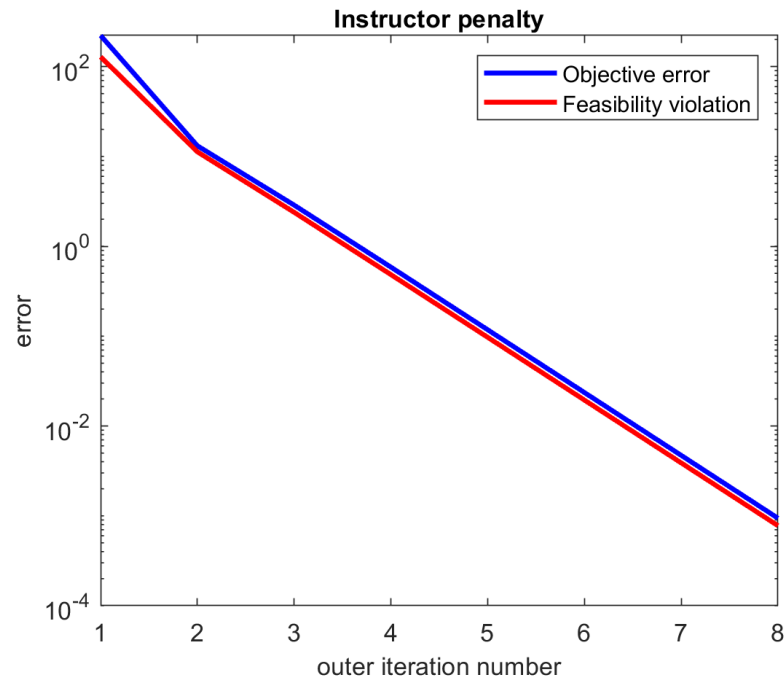


Figure 2: Instructor Quadratic Penalty test_ALM_qp.m Result

Instructor Augmented Lagrangian Method Figure Result

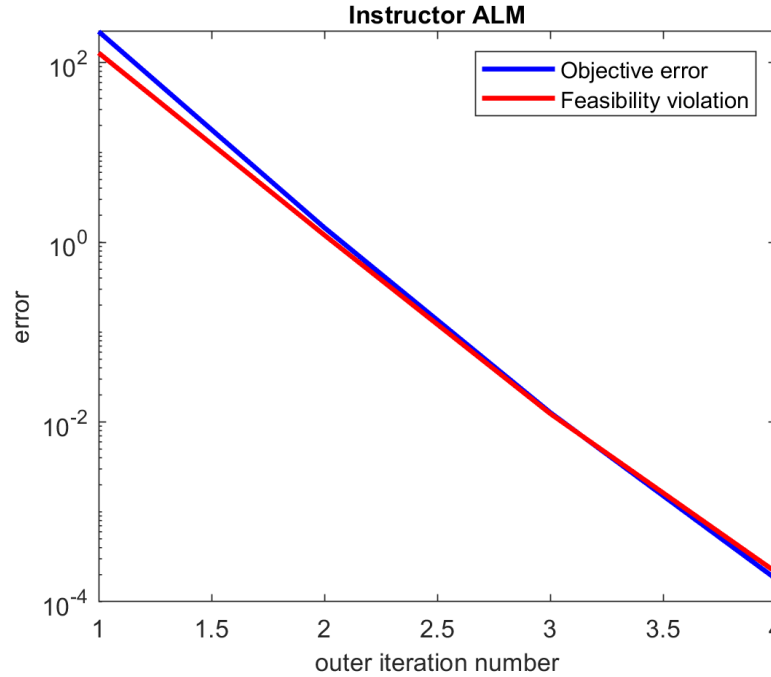


Figure 3: Instructor ALM method test_ALM_qp.m Result

Therefore, based on the plots of error vs iterations, we have nearly identical performances of functions, so we conclude our ALM_qp function is working correctly. Our student solver performs slightly slower than the instructor's, likely due to MatLab implementations.

In comparison to the instructor's quadratic penalty method, the ALM method perform much faster and efficiently, in terms of both speed and iterations. From the plot, we see the ALM methods takes 4 iterations to achieve objective error and feasibility violations of around 10^{-4} . Meanwhile, the quadratic penalty method takes double the iterations (8) to achieve the same error level. Further, when looking at the runtimes in the test output, the ALM methods take between 4 and 5 seconds, and the quadratic penalty method takes just over 100 seconds. This is a huge increase in speed. Thus, clearly the ALM method completely outperforms the quadratic penalty for this problem.

Problem 2

Consider the Least Absolute Deviation (LAD) problem:

$$\underset{\mathbf{x} \in \mathbb{R}^n}{\text{minimize}} \|\mathbf{Ax} - \mathbf{b}\|_1 \quad (2)$$

where $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{b} \in \mathbb{R}^m$ are given. The LAD problem is equivalent to

$$\underset{\mathbf{x} \in \mathbb{R}^n, \mathbf{z} \in \mathbb{R}^m}{\text{minimize}} \|\mathbf{z}\|_1, \text{ s.t. } \mathbf{Ax} - \mathbf{z} - \mathbf{b} = \mathbf{0}. \quad (3)$$

The augmented Lagrangian function of (3) is

$$\mathcal{L}_\rho(\mathbf{x}, \mathbf{z}, \mathbf{y}) = \|\mathbf{z}\|_1 + \mathbf{y}^\top (\mathbf{Ax} - \mathbf{z} - \mathbf{b}) + \frac{\rho}{2} \|\mathbf{Ax} - \mathbf{z} - \mathbf{b}\|^2,$$

where $\rho > 0$ is the penalty parameter, and $\mathbf{y} \in \mathbb{R}^m$ is the multiplier vector. The iterative update schemes of ADMM on solving (3) is

$$\mathbf{x}^{k+1} = \arg \min_{\mathbf{x} \in \mathbb{R}^n} \mathcal{L}_\rho(\mathbf{x}, \mathbf{z}^k, \mathbf{y}^k), \quad (4a)$$

$$\mathbf{z}^{k+1} = \arg \min_{\mathbf{z} \in \mathbb{R}^m} \mathcal{L}_\rho(\mathbf{x}^{k+1}, \mathbf{z}, \mathbf{y}^k), \quad (4b)$$

$$\mathbf{y}^{k+1} = \mathbf{y}^k + \rho(\mathbf{Ax}^{k+1} - \mathbf{z}^{k+1} - \mathbf{b}) \quad (4c)$$

1. Assume $\mathbf{A}$ has column full rank, i.e., $\mathbf{A}^\top \mathbf{A}$ is nonsingular. Give the closed-form solution of $\mathbf{x}^{k+1}$ in the update (4a). The solution should involve the inverse of $\mathbf{A}^\top \mathbf{A}$.

First, we note our rewritten LAD problem

$$\min_{x \in \mathbb{R}^n, z \in \mathbb{R}^n} \|z\|_1, \quad \text{s.t. } Ax - z - b = 0$$

is in the form

$$\min_{x \in \mathbb{R}^n, z \in \mathbb{R}^n} f(x) + g(z) \quad \text{s.t. } Ax + Bz = c$$

where

$$\begin{aligned} f(x) &= 0 \\ g(z) &= \|z\|_1 \\ B &= -I \\ c &= b \end{aligned}$$

Now, we have the augmented Lagrangian function

$$\mathcal{L}_\rho(\mathbf{x}, \mathbf{z}, \mathbf{y}) = \|\mathbf{z}\|_1 + \mathbf{y}^\top (\mathbf{A}\mathbf{x} - \mathbf{z} - \mathbf{b}) + \frac{\rho}{2} \|\mathbf{A}\mathbf{x} - \mathbf{z} - \mathbf{b}\|^2,$$

With updates

$$\begin{aligned}\mathbf{x}^{k+1} &= \arg \min_{\mathbf{x} \in \mathbb{R}^n} \mathcal{L}_\rho(\mathbf{x}, \mathbf{z}^k, \mathbf{y}^k) \\ \mathbf{z}^{k+1} &= \arg \min_{\mathbf{z} \in \mathbb{R}^m} \mathcal{L}_\rho(\mathbf{x}^{k+1}, \mathbf{z}, \mathbf{y}^k) \\ \mathbf{y}^{k+1} &= \mathbf{y}^k + \rho(\mathbf{A}\mathbf{x}^{k+1} - \mathbf{z}^{k+1} - \mathbf{b})\end{aligned}$$

Now solving

$$\begin{aligned}x^{k+1} &= \arg \min_{x \in \mathbb{R}^n} \mathcal{L}_\rho(\mathbf{x}, \mathbf{z}^k, \mathbf{y}^k) \\ &= \arg \min_{x \in \mathbb{R}^n} \left(\|z^k\|_1 + (y^k)^\top (Ax - z^k - b) + \frac{\rho}{2} \|Ax - z^k - b\|^2 \right)\end{aligned}$$

Now, we can drop the arguments that are constant relative to x

$$\implies x^{k+1} = \arg \min_{x \in \mathbb{R}^n} \left((y^k)^\top (Ax) + \frac{\rho}{2} \|Ax - z^k - b\|^2 \right)$$

Now taking gradient = 0

$$\nabla_x \mathcal{L} = A^\top y^k + \rho A^\top (Ax - z^k - b) = 0$$

$$\implies A^\top y^k + \rho A^\top Ax - \rho A^\top (z^k + b) = 0$$

$$\implies \rho A^\top Ax = \rho A^\top (z^k + b) - A^\top y^k$$

$$\implies A^\top Ax = A^\top (z^k + b) - \frac{1}{\rho} A^\top y^k$$

$$\implies x^{k+1} = (A^\top A)^{-1} A^\top \left(z^k + b - \frac{1}{\rho} y^k \right)$$

Thus, we get our solution

$$x^{k+1} = (A^T A)^{-1} A^T \left(z^k + b - \frac{1}{\rho} y^k \right)$$

-
2. Recall that we have derived the proximal mapping of $\lambda \|\cdot\|_1$ for any $\lambda > 0$ in class. Let's denote it as $\mathbf{prox}_{\lambda \|\cdot\|_1}$. Use the proximal mapping to derive a closed form solution of $\mathbf{z}^{k+1}$ in the update (4b). In the solution, λ should be $1/\rho$.

First, we have

$$z^{k+1} = \arg \min_{z \in \mathbb{R}^m} \mathcal{L}_\rho(\mathbf{x}^{k+1}, \mathbf{z}, \mathbf{y}^k)$$

$$z^{k+1} = \arg \min_{z \in \mathbb{R}^m} \left(\|z\|_1 + (y^k)^T (Ax^{k+1} - z - b) + \frac{\rho}{2} \|Ax^{k+1} - z - b\|^2 \right)$$

Now, we can drop the arguments that are constant relative to z

$$\implies z^{k+1} = \arg \min_{z \in \mathbb{R}^m} \left(\|z\|_1 - (y^k)^T z + \frac{\rho}{2} \|Ax^{k+1} - z - b\|^2 \right)$$

Completing the square

$$\implies z^{k+1} = \arg \min_{z \in \mathbb{R}^m} \left(\|z\|_1 + \frac{\rho}{2} \left\| z - \left(Ax^{k+1} - b + \frac{1}{\rho} y^k \right) \right\|^2 \right)$$

Dividing by ρ to get equation into standard proximal mapping form

$$\implies z^{k+1} = \arg \min_{z \in \mathbb{R}^m} \left(\frac{1}{\rho} \|z\|_1 + \frac{1}{2} \left\| z - \left(Ax^{k+1} - b + \frac{1}{\rho} y^k \right) \right\|^2 \right)$$

Now using $\lambda = \frac{1}{\rho}$, the closed-form solution via the proximal operator is

$$\boxed{z^{k+1} = \mathbf{prox}_{\frac{1}{\rho} \|\cdot\|_1} \left(Ax^{k+1} - b + \frac{1}{\rho} y^k \right)}$$

-
3. Suppose $(\mathbf{x}^{k+1}, \mathbf{z}^{k+1}, \mathbf{y}^{k+1})$ is the output. At this point, give the violation of primal and dual feasibility.

First computing the violation of primal feasibility (denoted r^k)

From the class lecture we have

$$r^{k+1} = Ax^{k+1} + Bz^{k+1} * c$$

With $b = -I$, $c = b$

$$r^{k+1} = Ax^{k+1} - Iz^{k+1} - b = Ax^{k+1} - z^{k+1} - b$$

Thus we get our primal violation

$$\boxed{r^{k+1} = Ax^{k+1} - z^{k+1} - b}$$

Now computing the violation of dual feasibility (denoted s^k)

From our notes,

$$s^{k+1} = \rho A^T B (z^{k+1} - z^k)$$

With $B = -I$

$$s^{k+1} = -I\rho A^T (z^{k+1} - z^k) = -\rho A^T (z^{k+1} - z^k) = \rho A^T (z^k - z^{k+1})$$

Thus we get our dual feasibility violation of

$$\boxed{s^{k+1} = \rho A^T (z^k - z^{k+1})}$$

or equivalently

$$\boxed{s^{k+1} = -\rho A^T (z^{k+1} - z^k)}$$

-
4. Use the instructor's provided file `ADMM_LAD.m` to write a Matlab function `ADMM_LAD` with input **A**, **b**, initial vector **x0**, stopping tolerance **tol**, maximum number of iterations **maxit**, and penalty parameter $\rho > 0$. Also test your function by running the provided test file `test_ADMM_LAD.m` and compare to the instructor's function. Print your code and the results you get.

Alternating Direction Method of Multipliers implementation

```
function [x,hist_obj, out] = ADMM_LAD(A,b,rho,tol,maxit,x0)

% alternating direction method of multipliers for
% min_x ||A*x-b||_1

[m,n] = size(A);

% initialization
x = x0;
z = A*x - b;

hist_obj = norm(z,1);

% initialize the multiplier
y = zeros(m,1);

pr = 0;
dr = 1;

iter = 1;

% Compute transpose to avoid per-loop computations
At = A';
AtA = At*A;

while max(pr,dr) > tol && iter < maxit
    iter = iter + 1;

    % update x
```

```

x = AtA \ (At * (z + b - y/rho));

% update z
z0 = z;

% Compute Ax once
Ax = A*x;

% Initialize v and kappa
v = Ax - b + y/rho;
kappa = 1/rho;

% Soft-thresholding operator for the L1 norm
z = sign(v) .* max(abs(v) - kappa, 0);

% update y
y = y + rho * (Ax - z - b);

% compute primal and dual residual
pr = norm(Ax - z - b);
dr = norm(rho * A' * (z0 - z));

% compute and save objective value ||A*x-b||_1
obj = norm(Ax - b, 1);
hist_obj = [hist_obj; obj];

end
out.iter = iter;
out.pr = pr;
out.dr = dr;
end

```

Code Results: **test_ALM_qp.m** Output

```
>> test_ADMM_LAD
```

Optimal solution found.

Student solver: Total running time of instructor code is 0.1952

Instructor solver: Total running time of instructor code is 0.1100

Student Figure Result

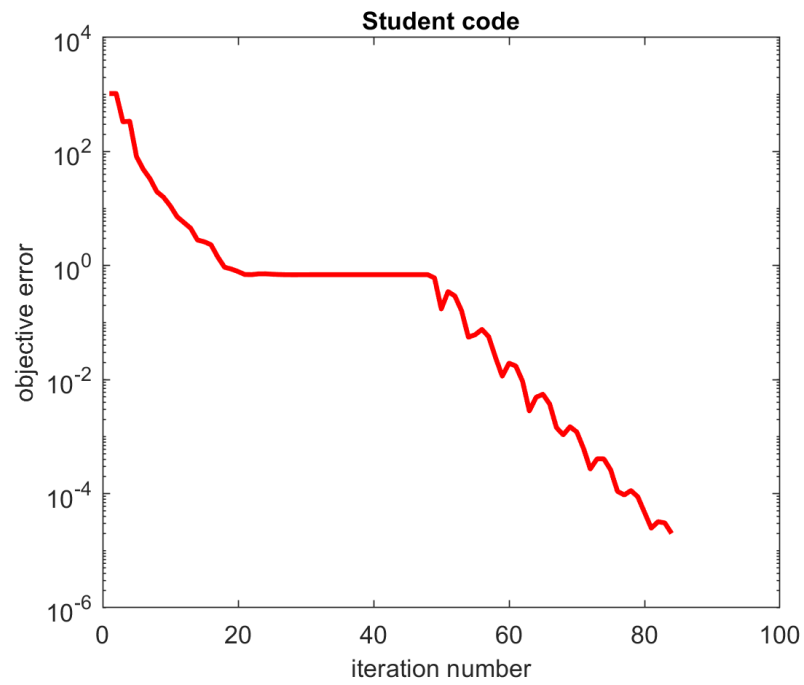


Figure 4: Student test_ADMM_LAD.m Result

Instructor Figure Result

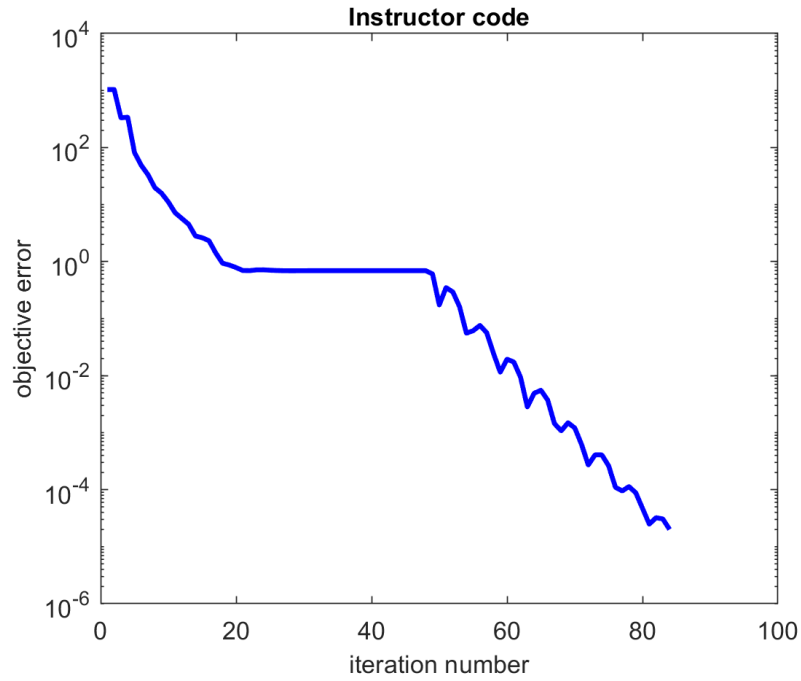


Figure 5: Instructor test_ADMM_LAD.m Result

Therefore, based on the plots of error vs iterations, we have nearly identical performances of functions, so we conclude our ADMM_LAD function is working correctly. Our student solver performs slightly slower than the instructor's, likely due to MatLab implementations.